



ENTERPRISE AI ARCHITECTURE

Architecting Enterprise RAG

A Technical Introduction of Retrieval-Augmented Generation (RAG).



The Challenge of Static LLMs in the Enterprise

LLM Constraints

Knowledge Gaps & Hallucinations

Large Language Models (LLMs) face significant limitations in enterprise settings. They suffer from a strict knowledge cutoff, meaning they lack awareness of recent events.

Furthermore, they do not have access to proprietary, private enterprise data. This often leads to hallucinations, where the model confidently generates incorrect or fabricated information.

Architectural Solution

RAG bridges this gap by grounding the model in verifiable, organization-specific data without requiring costly retraining or fine-tuning.

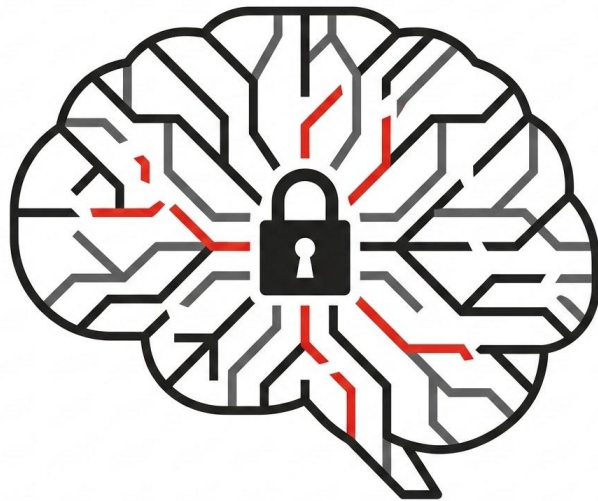


Figure 1.0: Isolated LLM Enterprise Boundary (Static / Locked Knowledge)

RAG System Architecture Overview

The Retrieval-Augmented Generation pipeline consists of three core phases.

01 / INGESTION

Ingestion Phase

First, the Ingestion phase processes and stores enterprise documents.

02 / RETRIEVAL

Retrieval Phase

Second, the Retrieval phase intercepts user queries to fetch semantically relevant context from the stored data.

03 / GENERATION

Generation Phase

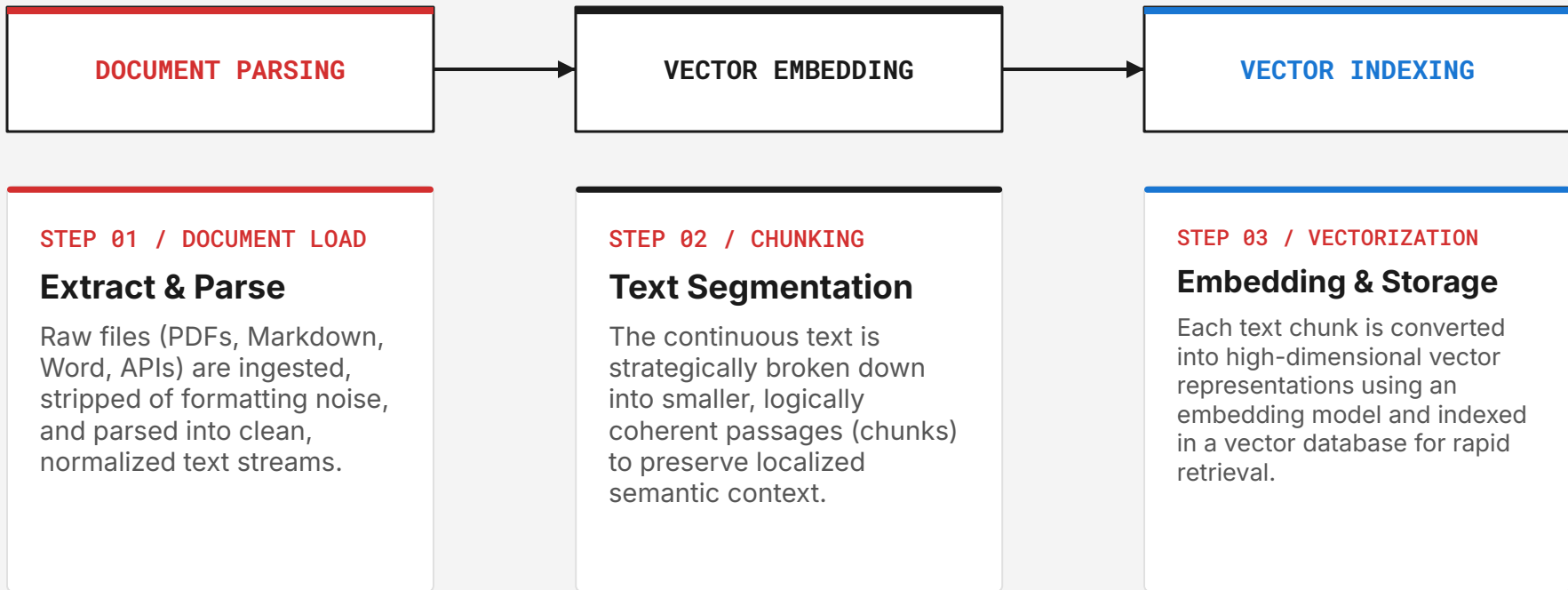
Finally, the Generation phase combines the original query with the retrieved context, prompting the LLM to synthesize an accurate, grounded response.

→ This end-to-end flow ensures data remains private while enhancing output accuracy.





Data Ingestion Pipeline



→ Quality at the ingestion phase directly determines the relevance and precision of subsequent retrieval steps.



Data Ingestion: Chunking, Embedding & Vectorization

01 / OVERLAPPING CHUNKS

Sliding Window Partitioning

Splits raw documents into overlapping segments to preserve semantic context across chunk boundaries.

CHUNK A

"...company policy on parental leave allows up to 12 weeks of paid..."

CHUNK B (20% OVERLAP)

"...parental leave allows up to 12 weeks of paid leave starting from..."

02 / VECTOR EMBEDDING

Semantic Representation

Passes text segments through an embedding model to convert natural language into mathematical coordinates.

EMBEDDING MODEL

Transforms text tokens into high-dimensional space where similar meanings cluster together.

VECTOR OUTPUT

[0.012, -0.284, 0.913, 0.054, -0.118, 0.442, 0.812, -0.091, 0.321...]

03 / VECTOR DATABASE

Storage & Indexing

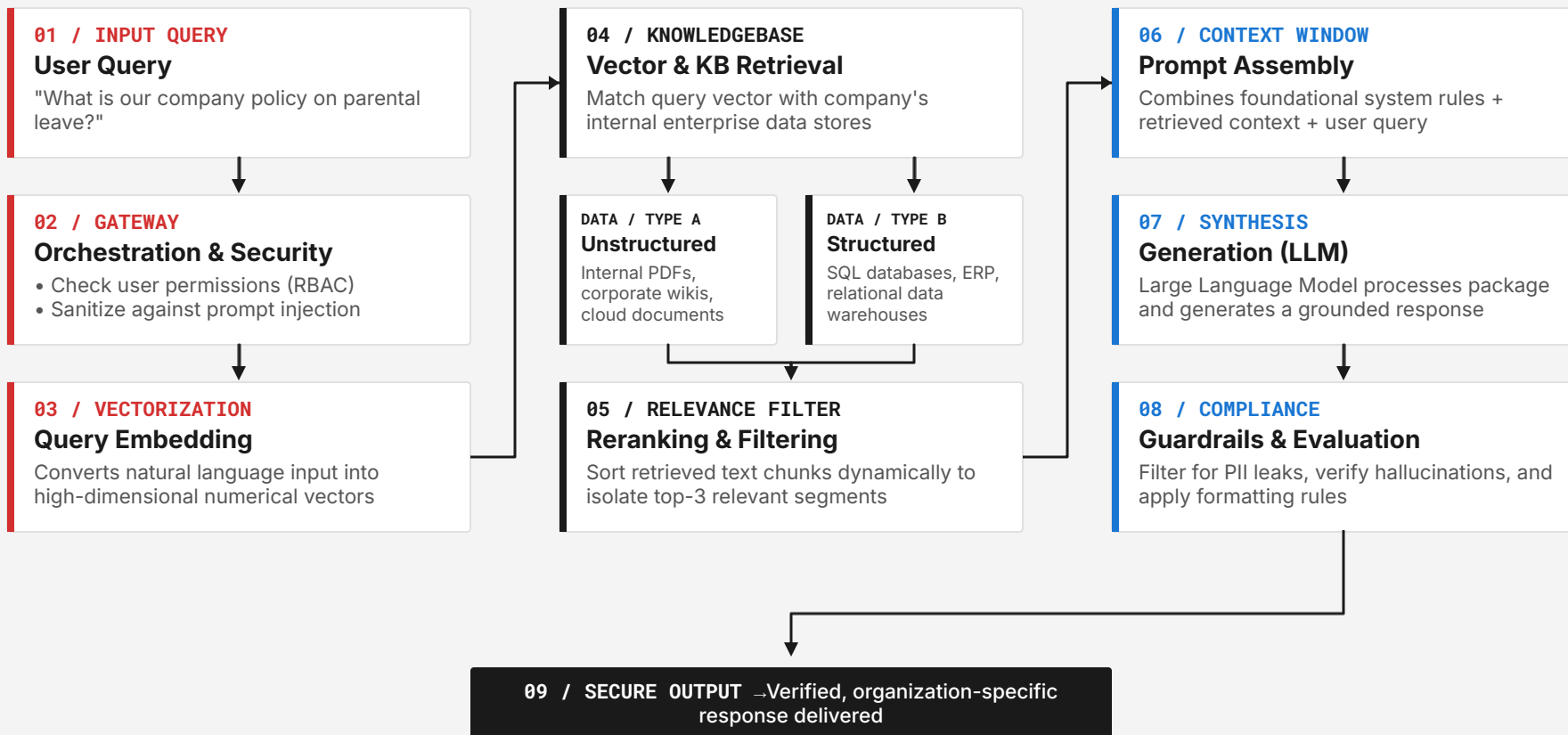
Stores high-dimensional vectors in specialized index structures for ultra-fast similarity searches during retrieval.

Chunk A Chunk B Other

Vectors with similar meanings are stored close together in vector space.



The Enterprise RAG Retrieval & Generation Pipeline





The Retrieval Mechanism

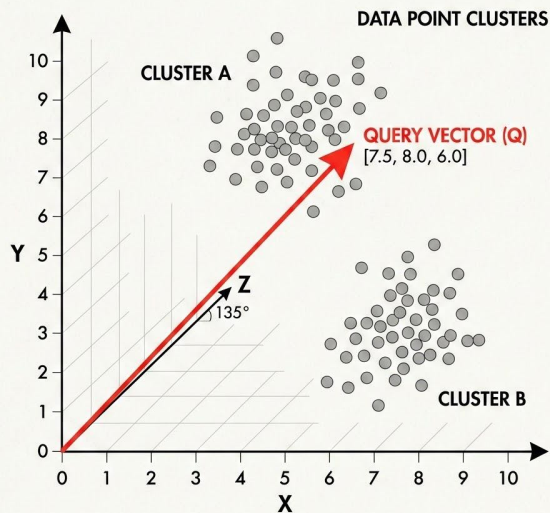
Query Processing

Semantic Vector Search

When a user submits a query, it is converted into a vector using the same embedding model used during ingestion. The system then performs a vector search against the database.

Using similarity metrics like Cosine Similarity or Euclidean Distance, it identifies the chunks closest to the query vector in the high-dimensional space. This semantic retrieval ensures the system fetches context based on meaning and intent, rather than just exact keyword matches.

3D COORDINATE SYSTEM (XYZ)





Enhancing Retrieval Precision

Stage 01 / Retrieval

Hybrid Search & Metadata Filtering

Basic vector search can be optimized for enterprise precision. Hybrid search combines semantic vector retrieval with traditional keyword scoring to capture both meaning and exact terminology.

Additionally, metadata filtering allows the system to restrict searches by date, author, or document type before the vector search occurs.

Stage 02 / Evaluation

Cross-Encoder Re-Ranking

Re-ranking models are then applied to evaluate and re-order the initial retrieved results based on deeper contextual relevance.



The Prompt Synthesis Phase

```
prompt_config.json

// Constructs structured dynamic prompt
{
  "system_instruction": "Use context strictly, cite sources.",
  "context": [
    "${top_ranked_retrieved_documents}"
  ],
  "query": "${user_original_query}",
  "constraints": [
    "prevent_hallucination",
    "strict_evidence_matching"
  ]
}
```

PHASE 03: SYNTHESIS

Augmented Prompt Template

In the final phase, the system constructs a dynamic prompt template. It injects the top-ranked retrieved documents as context alongside the user's original query.

System instructions are also appended, strictly guiding the LLM to base its answer solely on the provided context and to cite its sources.

The LLM then processes this enriched prompt to generate a synthesized, highly accurate, and organization-specific response.



Securing Enterprise Data

Role-Based Access Control (RBAC)

Implementing Role-Based Access Control (RBAC) ensures users only retrieve context from documents they are authorized to view, typically handled via metadata filtering at the database level.

Data Masking

Data masking techniques can anonymize sensitive information before it reaches the LLM.

Private Endpoints

Utilizing private network endpoints and enterprise-tier LLM APIs guarantees that proprietary data is never used to train public models.



Evaluating RAG Performance

Measuring RAG quality requires specialized evaluation frameworks.

01 / FAITHFULNESS

Faithfulness

Checks if the generated answer is strictly derived from the retrieved context without hallucinations.

02 / RELEVANCE

Answer Relevance

Ensures the response directly addresses the user's prompt.

03 / RECALL

Context Recall

Evaluates whether the retrieval system successfully fetched all the necessary information required to answer the query completely.



Production Challenges & Mitigations

Operational Hurdles

Performance Bottlenecks

Scaling RAG introduces several operational hurdles. High latency from sequential embedding, retrieval, and generation steps can be mitigated using semantic cache systems to serve frequent queries instantly.

Latency & Cost Optimization

API costs can be optimized by routing simpler queries to smaller, faster models.

Dynamic Updates & Indexing

Handling dynamic data updates requires efficient incremental indexing strategies to ensure the vector database remains synchronized with live enterprise repositories without requiring full rebuilds.



Key References & Sources

Authoritative academic papers, industry guides, and official technical documentation for production RAG.

01 / FOUNDATIONAL PAPER

Lewis, P., et al. (2020)

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

arxiv.org/abs/2005.11401

02 / INDUSTRY STANDARD

Prompt Engineering Guide

Retrieval Augmented Generation (RAG) for LLMs

promptingguide.ai/research/rag

03 / CLOUD PROVIDER

Google AI for Developers

Retrieval Augmented Generation Overview

[\[link removed\]](#)

04 / GRAPH PARADIGM

Retrieval-Augmented Graphs

GraphRAG: Navigating Explicit Entity Relationships

arxiv.org/abs/2501.00309

05 / DEVELOPMENT PRACTICES

LangChain & LangGraph

RAG: From Theory to Production Implementation

[\[link removed\]](#)

06 / SYSTEMS MANAGEMENT

Google Cloud Vertex AI

Managing Enterprise-Scale RAG Corpora

[\[link removed\]](#)



Key References & Sources

Authoritative academic papers, industry guides, and official technical documentation for production RAG.

01 / GITHUB REPOSITORY

[Agent with RAG Example](#)

A reference implementation for document ingestion, vector indexing, and interactive querying with integrated interaction logging.

02 / LLM SERVING

[vLLM](#)

A high-throughput, low-latency engine optimized for efficient LLM serving and real-time inference orchestration.

03 / VECTOR DATABASE

[Qdrant](#)

A production-grade vector database optimized for high-performance payload management, fast search, and scalable indexing.

04 / LLM APP PLATFORM

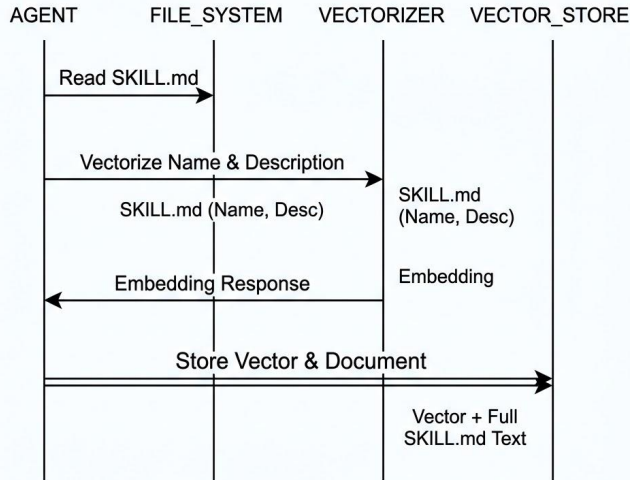
[Dify](#)

An open-source orchestration platform integrating AI workflows, production RAG pipelines, agent capabilities, and native observability.

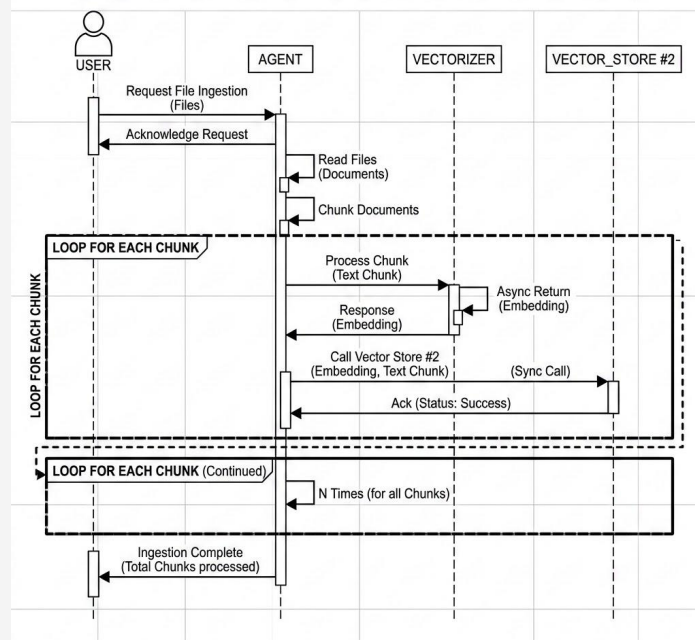
Addendum: Agent with RAG Ingestion Process

Diagram of the [SKILL.md](#) file Ingestion vs other documents

AI Agent - Document : SKILL.md



LADDER DIAGRAM: DOCUMENT INGESTION & VECTOR STORAGE PROCESS





Addendum: Agent with RAG Retrieval Process

Sequence Ladder Diagram: Detailed Step-by-Step Retrieval Workflow

